

SLIDE 1 — Declaring a Pointer

Definition:

Pointer declaration tells C that a variable will store the **address of a particular data type**.

Syntax

```
data_type *pointer_name;
```

Examples

```
int *p;  
float *f;  
char *c;  
double *d;
```

Meaning

```
int *p;
```

int → type of data p points to

* → p is a pointer

p → pointer variable name



Remember:

`int *p;` means **p can store the address of an int variable**.

SLIDE 2 — Initializing a Pointer

Declaring a pointer only creates the pointer variable.

Initialization gives it a valid address.

Example

```
int x = 10;  
int *p = &x;
```

```
printf("Value = %d", *p);
```

Output

Value = 10

Step-by-step

Step 1:

```
x = 10
```

Step 2:

```
&x = address of x
```

Step 3:

```
p = &x
```

Step 4:

```
*p = value stored at that address  
    = 10
```

 **Remember:**

Declare → Get Address → Store Address → Dereference

SLIDE 3 — Understanding **&** and *****

Two operators are extremely important.

& → **Address-of operator**

```
int x = 25;
```

```
printf("%p", (void*)&x);
```

&x gives the **memory address** of **x**.

***** → **Dereference operator**

```
int x = 25;
```

```
int *p = &x;
```

```
printf("%d", *p);
```

***p** gives the **value stored at that address**.

Quick View

&

Variable → Address → Value

x	p	*p
25	address	25

⚠ Important:

The meaning of ***** depends on where it is used.

```
int *p;    // declaration
*p = 50;   // dereferencing
```

SLIDE 4 — Different Ways to Initialize

1. Initialize during declaration

```
int x = 20;
int *p = &x;
```

2. Declare first, initialize later

```
int x = 20;
int *p;
```

```
p = &x;
```

Both are valid.

3. Initialize with **NULL**

If a pointer doesn't currently point to a valid object:

```
int *p = NULL;
```

This means the pointer intentionally points to **no valid object**.

```
p
↓
NULL
```

⚠ Never dereference a **NULL** pointer:

```
*p = 10; // ❌ Wrong
```

Best beginner habit

```
int x = 10;  
int *p = &x;
```

Initialize pointers before using them.

SLIDE 5 — Pointer Type Must Match

A pointer should normally point to a variable of the **corresponding type**.

Correct

```
int x = 10;  
int *p = &x;  
char ch = 'A';  
char *p = &ch;  
float f = 5.5;  
float *p = &f;
```

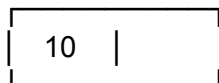
Why type matters?

The pointer type tells C how to interpret the memory when you dereference it and is also important for pointer arithmetic.

Memory Picture

```
int x = 10;
```

Address



int *p



Remember:

Pointer type should match the type of data it points to.

DEEP CONCEPT — What Actually Happens?

Consider:

```
int x = 100;  
int *p = &x;
```

Think of memory like numbered boxes:

Address	Value
---------	-------

1000 →	100
--------	-----

Suppose `x` is stored at address `1000`.

Then:

<code>x</code>	→ 100
<code>&x</code>	→ 1000

<code>p</code>	→ 1000
<code>*p</code>	→ 100

So:

```
printf("%d", x); // 100  
printf("%d", *p); // 100
```

Both access the **same value**, but through different paths.

The key relationship

	<code>&</code>		<code>*</code>	
<code>x</code>	→	<code>p</code>	→	<code>x</code>

More precisely:

`p == &x`

and

`*p == x`

when `p` points to `x`.



QUESTIONS

Q1. MCQ

What does this declaration mean?

```
int *p;
```

- A) **p** stores an integer value
- B) **p** stores the address of an integer
- C) **p** stores a character
- D) **p** stores a float

Hint: * indicates a pointer declaration.

Glossary: Pointer → variable that stores an address.

Answer: B

Q2. Fill in the blank

Complete the code:

```
int x = 50;  
int *p = _____;  
  
printf("%d", *p);
```

Hint: You need the address of **x**.

Glossary: & → address-of operator.

Answer: &x

Q3. What is the output?

```
int x = 25;  
int *p = &x;
```

```
*p = 40;
```

```
printf("%d", x);
```

- A) 25
- B) 40
- C) Address of `x`
- D) Error

 **Hint:** `*p` refers to `x`.

 **Glossary:** Dereference → access the value through a pointer.

 **Answer:** B) 40

Q4. Identify the error

```
int x = 10;
```

```
float *p = &x;
```

Why is this incorrect?

- A) `x` cannot have an address
- B) Pointer type doesn't match the data type
- C) `&` cannot be used
- D) `float` is invalid

 **Hint:** Compare `int` and `float`.

 **Glossary:** Data type → type of value stored by a variable.

 **Answer:** B

Q5. True or False

```
int *p;
```

automatically makes `p` point to a valid integer variable.

 **Hint:** Declaration and initialization are different.

 **Glossary:** Initialization → giving a variable its initial value.

✓ Answer: False

Q6. What does **NULL** mean?

```
int *p = NULL;
```

- A) **p** contains 0 as an integer value
- B) **p** points to no valid object
- C) **p** contains garbage intentionally
- D) **p** points to **NULL** variable

💡 **Hint:** **NULL** represents an invalid/no-object pointer value.

📖 **Glossary:** **NULL** → null pointer constant used to indicate no valid target.

✓ Answer: B

Q7. Arrange the steps

For:

```
int x = 10;  
int *p = &x;  
printf("%d", *p);
```

Arrange:

- A. Dereference **p**
- B. Declare variable **x**
- C. Store address in **p**
- D. Get address using **&**

💡 **Hint:** Start with creating **x**.

📖 **Glossary:** Dereference → obtain the value through the pointer.

✓ Answer: B → D → C → A